

Assignment – 13.5

Name: A.jeevan sai

H.no: 2303A51420

Batch: 21

Task Description #1 (Refactoring – Removing Global Variables) •

Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1 def calculate_interest(amount): return amount *
```

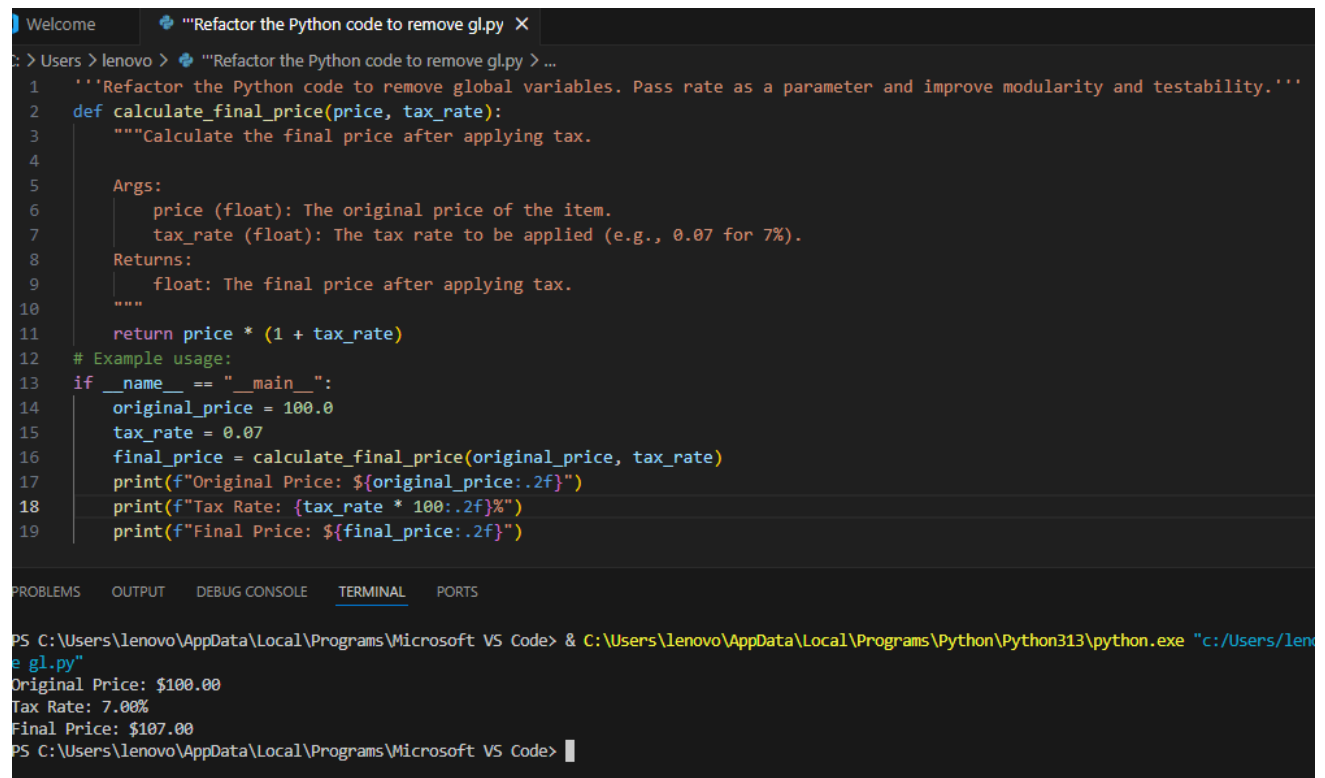
```
rate print(calculate_interest(1000)) • Expected Output: o
```

Refactored version passing rate as a parameter or using a configuration structure.

Prompt:

Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.

Code:



```
1  '''Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.'''
2  def calculate_final_price(price, tax_rate):
3      """Calculate the final price after applying tax.
4
5      Args:
6          price (float): The original price of the item.
7          tax_rate (float): The tax rate to be applied (e.g., 0.07 for 7%).
8      Returns:
9          float: The final price after applying tax.
10     """
11     return price * (1 + tax_rate)
12 # Example usage:
13 if __name__ == "__main__":
14     original_price = 100.0
15     tax_rate = 0.07
16     final_price = calculate_final_price(original_price, tax_rate)
17     print(f"Original Price: ${original_price:.2f}")
18     print(f"Tax Rate: {tax_rate * 100:.2f}%")
19     print(f"Final Price: ${final_price:.2f}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lenovo/AppData/Local/Programs/Microsoft VS Code/e gl.py"
Original Price: $100.00
Tax Rate: 7.00%
Final Price: $107.00
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code>
```

Observation:

- Global variable removed.
- Function is now reusable and testable.
- Follows modular programming principles.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas: o Readability o Logical simplification o Maintainability Legacy Code: score = 78 if score >= 90: print("Excellent") else: if score >= 75: print("Very Good") else: if score >= 60: print("Good") else:

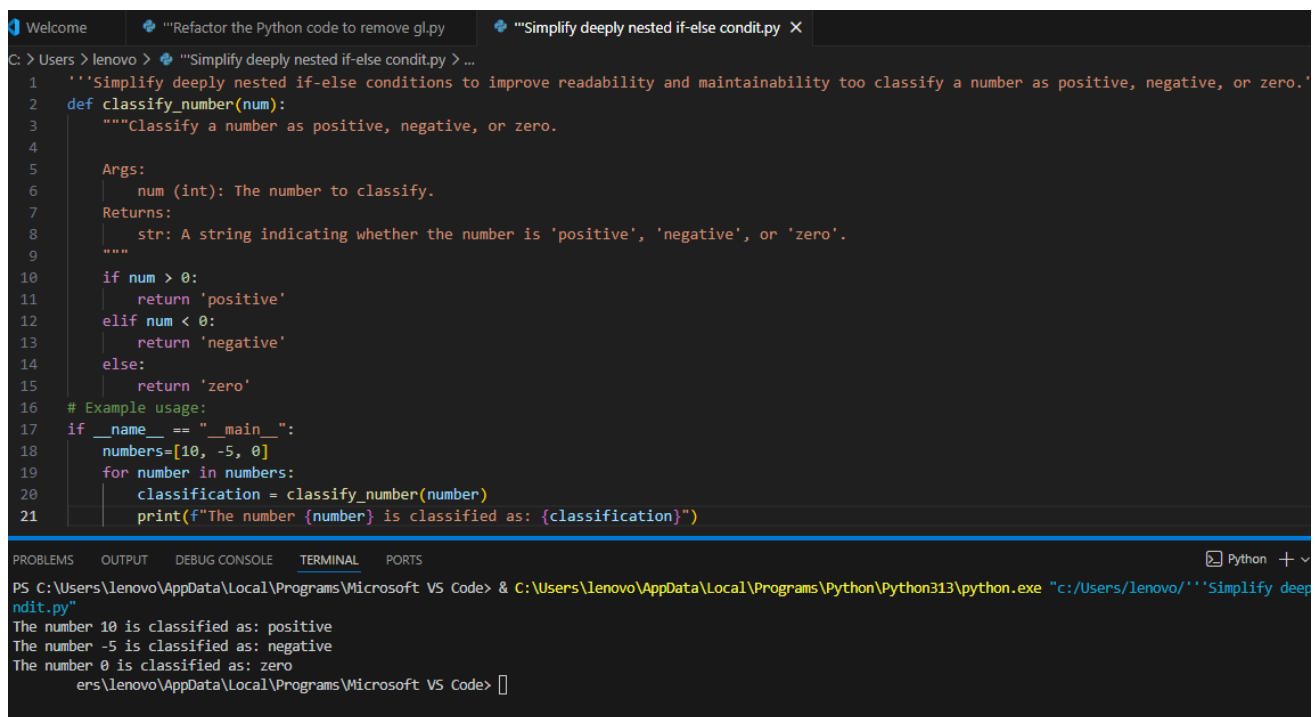
print("Needs Improvement") Expected

Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.

Prompt:

Simplify deeply nested if-else conditions to improve readability and maintainability. **Code:**



```
1 '''Simplify deeply nested if-else conditions to improve readability and maintainability too classify a number as positive, negative, or zero.'
2 def classify_number(num):
3     """Classify a number as positive, negative, or zero.
4
5     Args:
6         num (int): The number to classify.
7     Returns:
8         str: A string indicating whether the number is 'positive', 'negative', or 'zero'.
9     """
10    if num > 0:
11        return 'positive'
12    elif num < 0:
13        return 'negative'
14    else:
15        return 'zero'
16    # Example usage:
17    if __name__ == "__main__":
18        numbers=[10, -5, 0]
19        for number in numbers:
20            classification = classify_number(number)
21            print(f"The number {number} is classified as: {classification}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v

```
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lenovo/'''Simplify deep
ndit.py"
The number 10 is classified as: positive
The number -5 is classified as: negative
The number 0 is classified as: zero
ers\lenovo\AppData\Local\Programs\Microsoft VS Code> [
```

Observation:

- Removed nested structure.
- Improved readability.
- Easier to modify grade ranges.

Task 3 (Refactoring Repeated File Handling Code) • Task: Use

AI to refactor repeated file open/read/close logic.

- Focus Areas:
 - o DRY principle
 - o Context managers
 - o Function reuse
- Legacy Code: `f = open("data1.txt")`
`print(f.read())` `f.close()` `f = open("data2.txt")` `print(f.read())`
`f.close()`

Expected Outcome:

- o Reusable function using `with open()` and parameters.

Prompt:

Refactor repeated file open/close logic using context managers and reusable functions.

Code:

```
C: > Users > lenovo >  "Generate a python function to Refacto.py >  file_path
1  '''Generate a python function to Refactor repeated file open/close logic using context managers and reusable functions.'''
2  from pathlib import Path
3
4
5  def read_file(file_name):
6      """Read the contents of a file and print it.
7      Args:
8          file_name (str): The name of the file to read.
9      """
10     file_path = Path(__file__).resolve().parent / file_name
11     try:
12         with open(file_path, 'r', encoding='utf-8') as f:
13             print(f.read())
14     except FileNotFoundError:
15         print("File not found: {file_path}")
16
17
18 # Example usage
19 if __name__ == "__main__":
20     read_file("data1.txt")
21     read_file("data2.txt")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lenovo/'
acto.py"
File not found: {file_path}
File not found: {file_path}
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> |
```

Observation:

- Applied DRY principle.
- Used `with open()` (context manager).
- Prevents file leaks.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.
- Focus Areas: o Time complexity o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
        print("Access Granted")
    if not found:
        print("Access Denied")
```

Expected Outcome:

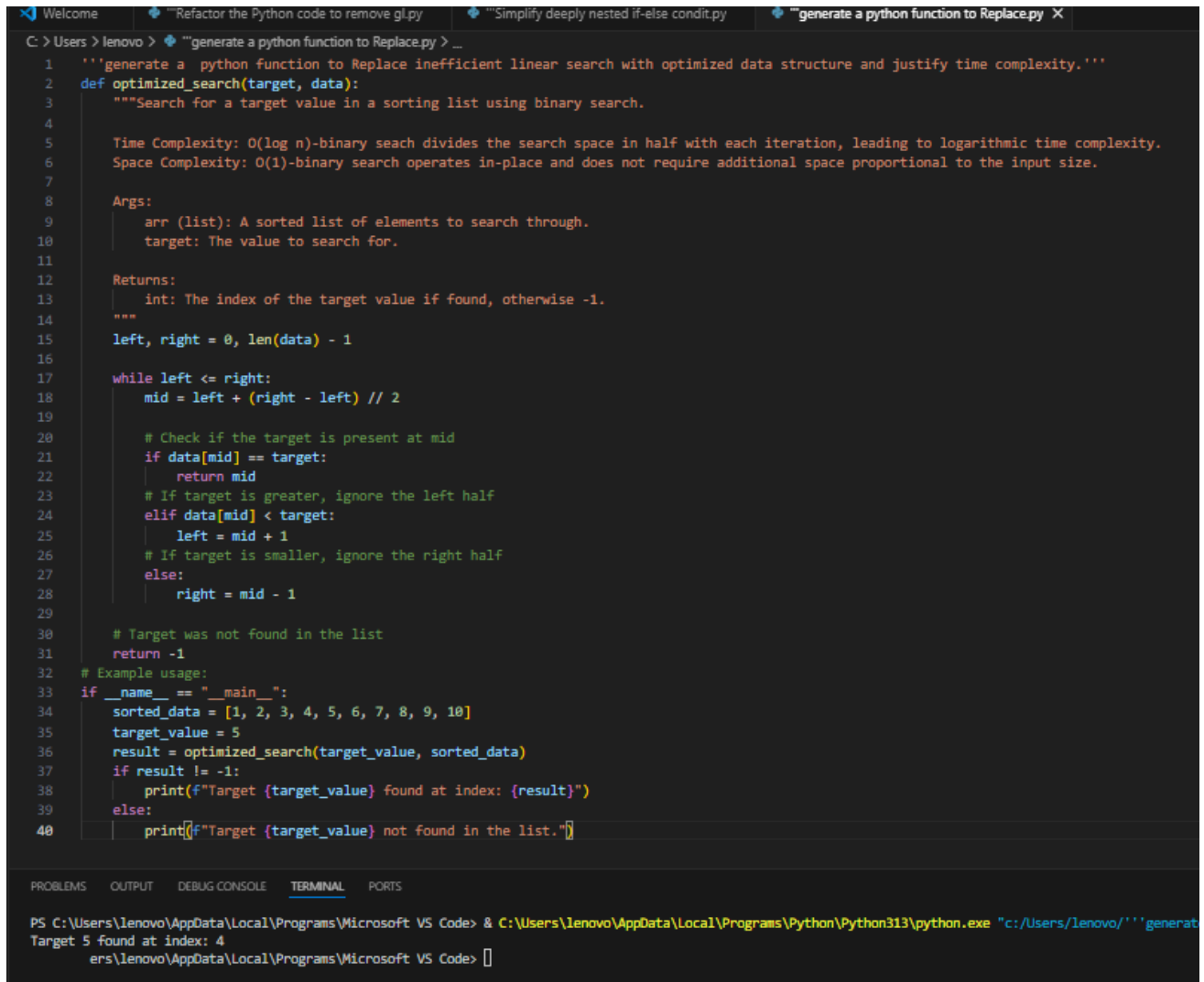
o Use of sets or dictionaries with complexity justification.

Prompt:

Replace inefficient linear search with optimized data structure and justify time complexity.

Code:

5



```
1 '''generate a python function to Replace inefficient linear search with optimized data structure and justify time complexity.'''
2 def optimized_search(target, data):
3     """Search for a target value in a sorting list using binary search.
4
5     Time Complexity: O(log n)-binary search divides the search space in half with each iteration, leading to logarithmic time complexity.
6     Space Complexity: O(1)-binary search operates in-place and does not require additional space proportional to the input size.
7
8     Args:
9         arr (list): A sorted list of elements to search through.
10        target: The value to search for.
11
12    Returns:
13        int: The index of the target value if found, otherwise -1.
14    """
15    left, right = 0, len(data) - 1
16
17    while left <= right:
18        mid = left + (right - left) // 2
19
20        # Check if the target is present at mid
21        if data[mid] == target:
22            return mid
23        # If target is greater, ignore the left half
24        elif data[mid] < target:
25            left = mid + 1
26        # If target is smaller, ignore the right half
27        else:
28            right = mid - 1
29
30    # Target was not found in the list
31    return -1
32
33 # Example usage:
34 if __name__ == "__main__":
35     sorted_data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
36     target_value = 5
37     result = optimized_search(target_value, sorted_data)
38     if result != -1:
39         print(f"Target {target_value} found at index: {result}")
40     else:
41         print(f"Target {target_value} not found in the list.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lenovo/'generat
Target 5 found at index: 4
ers\lenovo\AppData\Local\Programs\Microsoft VS Code> []

Observation:

- Changed list → set.

- Time complexity improved from **$O(n)$** to **$O(1)$** average case.
- More scalable for large datasets.

Task 5 (Refactoring Procedural Code into OOP Design) •

Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas: o Object-Oriented principles o Encapsulation

```
print(net)
```

Expected Outcome:

- o A class like `EmployeeSalaryCalculator` with methods and attributes.

Prompt:

Convert procedural salary calculation code into an object-oriented design with encapsulation.

Code:

```
C: > Users > lenovo >  ""Convert procedural salary calculation.py > ...
1  '''Convert procedural salary calculation code into an object-oriented design with encapsulation.'''
2  class Employee:
3      def __init__(self, name, base_salary):
4          self.name = name
5          self.base_salary = base_salary
6
7      def calculate_salary(self):
8          # Assuming some complex logic for salary calculation
9          return self.base_salary * 1.2 # Example: adding a 20% bonus
10 # Example usage
11 if __name__ == "__main__":
12     emp1 = Employee("Alice", 50000)
13     emp2 = Employee("Bob", 60000)
14     print(f"{emp1.name}'s salary: {emp1.calculate_salary()}")
15     print(f"{emp2.name}'s salary: {emp2.calculate_salary()}")
16
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe .\encapsulation.py

Alice's salary: 60000.0

Bob's salary: 72000.0

PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> 

0

0

0

● Update is ready. click to restart

Observation:

- Applied OOP principles.
- Encapsulation achieved.
- Reusable for multiple employees.

Task •

Task:

6 (Refactoring for Performance Optimization)

Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas: o

Algorithmic optimization o

Use of built-in functions

Legacy Code: total = 0 for i

in range(1, 1000000): if i %

2 == 0: total += i

print(total)

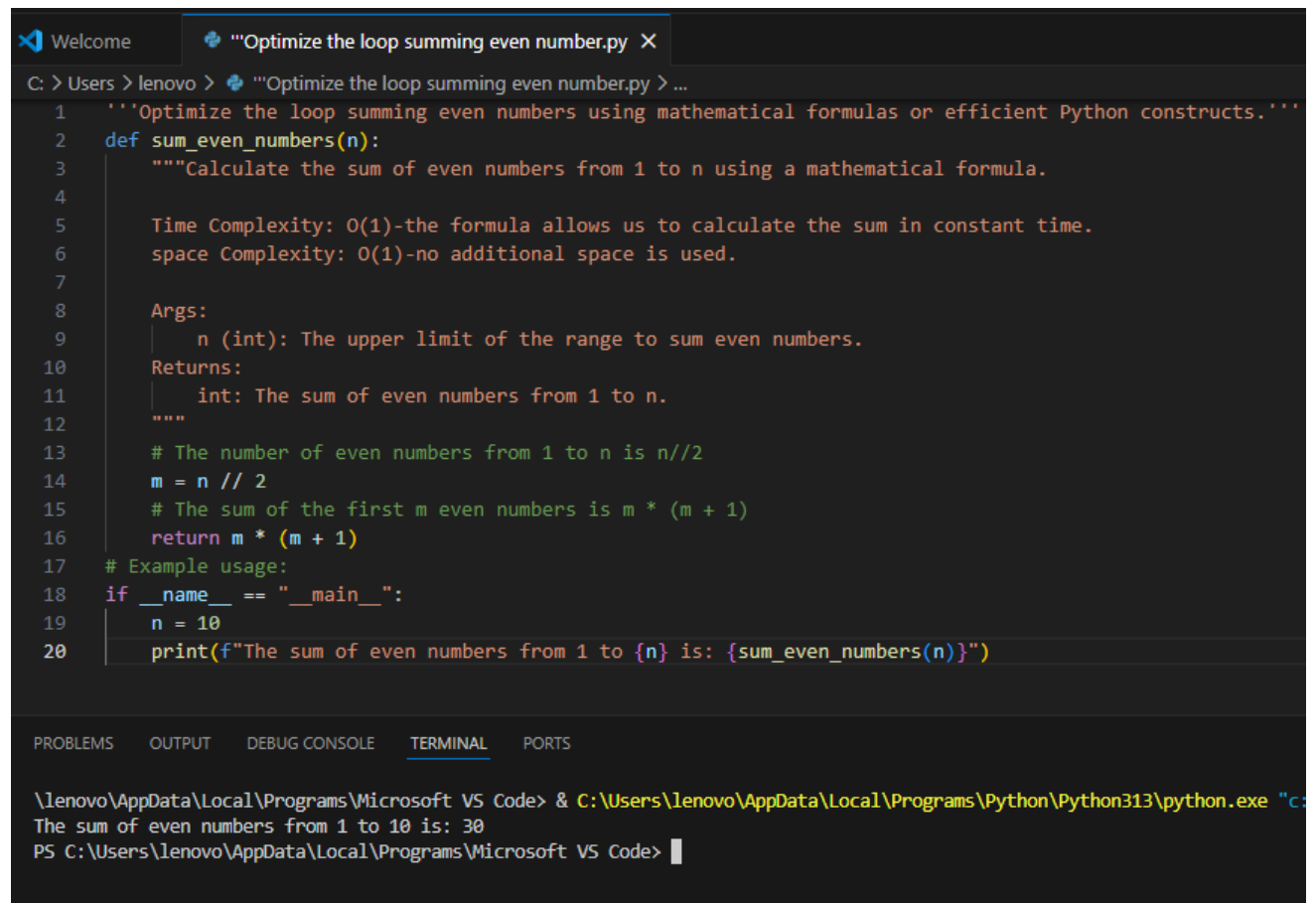
Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:

Optimize the loop summing even numbers using mathematical formulas or efficient Python constructs.

Code:



```
1 '''Optimize the loop summing even numbers using mathematical formulas or efficient Python constructs.'''
2 def sum_even_numbers(n):
3     """Calculate the sum of even numbers from 1 to n using a mathematical formula.
4
5     Time Complexity: O(1)-the formula allows us to calculate the sum in constant time.
6     Space Complexity: O(1)-no additional space is used.
7
8     Args:
9         n (int): The upper limit of the range to sum even numbers.
10    Returns:
11        int: The sum of even numbers from 1 to n.
12    """
13    # The number of even numbers from 1 to n is n//2
14    m = n // 2
15    # The sum of the first m even numbers is m * (m + 1)
16    return m * (m + 1)
17 # Example usage:
18 if __name__ == "__main__":
19     n = 10
20     print(f"The sum of even numbers from 1 to {n} is: {sum_even_numbers(n)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
\lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:
The sum of even numbers from 1 to 10 is: 30
PS C:\Users\lenovo\AppData\Local\Programs\Microsoft VS Code> |
```

Observation:

- Replaced $O(n)$ loop with $O(1)$ formula.
- Massive performance improvement.
- Suitable for large-scale computations.

Task •

Task:

7 (Removing Hidden Side Effects)

Refactor code that modifies shared mutable state.

- Focus Areas:
 - o Functional-style refactoring
 - o Predictability Legacy Code: `data = []`
`def add_item(x): data.append(x)`
`add_item(10) add_item(20) print(data)`
- Expected Outcome:
- o Refactored function returning values instead of mutating globals.

Prompt:

Refactor function to avoid modifying global mutable state and return new values instead.

Code:

```
ai123.py > ...
1 '''Refactor function to avoid modifying global mutable state and return new values instead.'''
2 def add_to_list(lst, item):
3     """Add an item to a list and return the new list.
4
5     Args:
6         lst (list): The original list.
7         item: The item to be added to the list.
8
9     Returns:
10        list: A new list with the item added.
11    """
12    new_list = lst.copy() # Create a copy of the original list to avoid modifying it
13    new_list.append(item) # Add the item to the new list
14    return new_list # Return the new list with the item added
15
16 # Example usage:
17 original_list = [1, 2, 3]
18 new_list = add_to_list(original_list, 4)
19 print("Original list:", original_list) # Output: Original list: [1, 2, 3]
20 print("New list:", new_list) # Output: New list: [1, 2, 3, 4]
```

PROBLEMS 134 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai123.py"
Original list: [1, 2, 3]
New list: [1, 2, 3, 4]
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Removed global mutation.
- Function is now pure and predictable.
- Improves testability. **8 (Refactoring Complex Input Validation Logic)**

Use AI to simplify and modularize complex validation

rules.

- Focus Areas:
 - o Readability
 - o Testability Legacy Code: `password = input("Enter password: ")` if

```
len(password) >= 8: if any(c.isdigit()
for c in password): if any(c.isupper()
for c in password): print("Valid
Password") else: print("Must contain
uppercase") else:
print("Must contain digit") else:
print("Password too short") Expected
Outcome:
```

o Separate validation functions with clear responsibility **Prompt:**

Modularize password validation rules into separate functions to improve readability and testability. **Code:**

```

1  """Modularize password validation rules into separate functions to improve readability and testability"""
2  def is_valid_length(password):
3      return len(password) >= 8
4  def has_uppercase(password):
5      return any(char.isupper() for char in password)
6  def has_lowercase(password):
7      return any(char.islower() for char in password)
8  def has_digit(password):
9      return any(char.isdigit() for char in password)
10 def has_special_char(password):
11     special_characters = "!@#$%^&*()-_+[]{}|;: '\",.<>?/"
12     return any(char in special_characters for char in password)
13 def validate_password(password):
14     """Validate the password based on defined rules.
15
16     Args:
17         password (str): The password to validate.
18
19     Returns:
20         bool: True if the password is valid, False otherwise.
21     """
22     if not is_valid_length(password):
23         print("Password must be at least 8 characters long.")
24         return False
25     if not has_uppercase(password):
26         print("Password must contain at least one uppercase letter.")
27         return False
28     if not has_lowercase(password):
29         print("Password must contain at least one lowercase letter.")
30         return False
31     if not has_digit(password):
32         print("Password must contain at least one digit.")
33         return False
34     if not has_special_char(password):
35         print("Password must contain at least one special character.")
36         return False
37     return True
38 # Example usage
39 if __name__ == "__main__":
40     password = input("Enter a password to validate: ")
41     if validate_password(password):

```

PROBLEMS (139) OUTPUT DEBUG CONSOLE TERMINAL (9) PORTS

```

PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ail
"
Enter a password to validate: Kushal1234
Password must contain at least one special character.
Password is invalid.
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ail
"
Enter a password to validate: Kushal@1234
Password is valid.
PS C:\Users\2303A\OneDrive\Desktop\python course>

```

Observation:

- Broke validation into small reusable functions.
- Improved readability.
- Easier to unit test each rule independently.